



FACULTY OF SCIENCES OF THE
UNIVERSITY OF PORTO

MASTER'S DEGREE IN INFORMATION SECURITY

Systems & Data Security

Public Ledger for Auctions over Kademlia

Marco Carneiro (up201908329)

Rui Fernandes (up202103071)

May, 2023

Contents

1	Introduction	1
2	Kademlia	1
2.1	Sybil & Eclipse Attacks	2
3	Public Ledger	2
3.1	Transactions	2
3.2	Consensus Mechanisms	3
4	Reputation System	3
5	Auction Mechanisms	4
6	Conclusion	4

List of Figures

Acronyms

DHT	Distributed Hash Table
DPoS	Delegated Proof-of-Stake
P2P	Peer-To-Peer
PoS	Proof-of-Stake
PoW	Proof-of-Work
RPC	Remote Procedure Call
UTXO	Unspent Transaction Output

1 Introduction

The purpose of this project is to develop and implement a decentralized public ledger capable of storing auction transactions. This implementation is divided into the following three parts:

- **P2P network:** The Peer-To-Peer (P2P) network should allow a node to join it and gossip the necessary data to other nodes in order to support the blockchain.
- **Decentralized Public Ledger:** The public ledger should be modular and be able to store auction transactions. As the consensus mechanism, we support both Proof-of-Work (PoW) and Proof-of-Stake (PoS).
- **Auction Mechanisms:** The auction system capable of supporting sellers and buyers using the a single attribute auction that follows the English auction.

The report is structured as follows: Section 2 describes the implementation of a Kademlia P2P network. Section 3 details the implementation of the ledger and the underlying consensus mechanisms. Section 4 discusses some challenges that come with the potential implementation of a reputation system. Section 5 describes the auction system. Finally, Section 6 concludes the report.

2 Kademlia

The P2P network uses the Kademlia [1] protocol for a Distributed Hash Table (DHT). This protocol relies on the ID each node to determine the distance between nodes, which is computed based on the XOR metric performed on their node IDs.

The Kademlia routing table consists of a collection of k -buckets, where each bucket contains a number of nodes are equidistant to the self node. The routing table is structured in such a way that the bucket at index n contains nodes with an n -bit prefix equal to the self node. Nodes within each k -bucket are sorted based on their last seen timestamp.

In addition to the four RPC of the Kademlia protocol, we implemented two additional RPCs: one to broadcast a message to all the nodes in the network, and one to send a message to a specific node. The Kademlia implementation follows its specification which dictates the usage of four main processes. These would allow the network to gossip the existence, status and distance of nodes within it.

The main four RPCs are: PING, STORE, FIND_NODE and FIND_VALUE. In order, these allow to check for the status of a node, store given information on nearest nodes, find a specific node on the network by replying which nodes are closest to the requested key, and to find a specific value in the storage of the recipient of the request. These RPCs calls were implemented using gRPC for remote communication and its underlying Netty abstracted implementation.

2.1 Sybil & Eclipse Attacks

To prevent Sybil attacks [2], when a new node joins the network, their node ID must be mined in a process similar to PoW that involves finding a nonce that, when combined with the node’s address and port, generates a hash that meets specific criteria [3]. This ensures that it is not computationally feasible for an attacker to flood the network with nodes under their control, as the process of joining the network requires a significant amount of work. By requiring new nodes to invest computational resources, such as CPU power or energy, it reinforces the authenticity of participating nodes.

In addition, the concurrency parameter α serves as an additional safety measure to mitigate Eclipse attacks. Essentially, an Eclipse attack occurs when a malicious entity gains control over a node’s network view, “isolating” it from genuine nodes and potentially manipulating the information exchanged within the network. By choosing an appropriate value for this parameter, the protocol increases the likelihood that genuine nodes will be queried during network operations – a higher value for α ensures that a more legitimate nodes are involved in the process. In this case, we consider the concurrency parameter $\alpha = 3$, as discussed in [1]. Overall, this measure serves as a safeguard against Eclipse attacks in the sense that reduces the risk of a single malicious entity having complete control over the network view of a specific node.

3 Public Ledger

The blockchain is essentially a chronological chain of interconnected blocks. Each block contains a number of transactions that may differ from the number of transactions in preceding or subsequent blocks

Subsequent blocks in the blockchain provide an additional layer of verification for preceding blocks. Each new block includes information that references the previous block, confirming its validity. This reinforces the reliability of the entire blockchain. In other words, the fact that a block references the previous one through its hash ensures that any blockchain is immutable and tamper-resistant.

3.1 Transactions

Once transactions are recorded in a block, they become an integral part of the blockchain [4]. These transactions are initially added to a transaction pool that contains pending transactions, which will then be used to generate a new block and add it to the ledger.

The concept of UTXOs plays a crucial role in maintaining the integrity and security of the blockchain. Each transaction output is considered unspent until it is consumed as an input in a subsequent transaction. Once a transaction output is utilized, it can no longer be used

as an input in any other transaction. This prevents the duplication or reuse of transaction outputs, safeguarding the blockchain against fraudulent activities such as double spending.

3.2 Consensus Mechanisms

In PoW, the process of mining a block consists in finding a nonce that results in a hash of the block header starting with a specific prefix length of zeros. This prefix length serves as the challenge for the block’s hash, setting the difficulty for mining the mining process. Miners iteratively try different nonce value until they find one that solves the challenge.

Due to timing constraints, we were not able to implement DPoS. Instead, we implemented PoS. In this case, we consider that all nodes in the network can act as validators. Then, when validating the transactions of a block, we choose a validator “at random”, taking into account their stake in the network – i.e. nodes with a higher stake are more likely to be chosen as validators.

4 Reputation System

Due to time limitations, we were not able to implement a reputation system. Regardless, our idea was to implement a system that allowed us to identify unreliable nodes within the network. As a consequence, these nodes would be used less often for routing and would potentially be ignored altogether.

To achieve this, we would compute and publish a **reputation** score to each node in the network, which would be then updated based on the behaviour of the corresponding node – deviating from the protocol would lead to a decrease in reputation, whereas adherence to the protocol would result in an increase in reputation. While implementing such a system may seem a simple task, it is by no means a trivial task due to several intricacies.

For example, there is the possibility of a node accumulating excessive positive reputation, which would potentially allow it to perform negative actions without a significant penalty. To address this issue, different approaches can be considered. We could, for example, implement a system where it becomes increasingly difficult to gain reputation for nodes with high and low reputation, which would make it harder to accumulate reputation and more challenging to recover from negative actions. On the other hand, we could also cap the maximum reputation value to prevent this issue. Finally, we could implement a system where the penalty for malicious actions increases with a higher reputation score. Ideally, a combination of these approaches should be considered.

In addition, such a reputation system should balance accuracy and efficiency. In other words, it should be able to accurately capture a node’s behavior while maintaining low overheads in terms of computation, communication and storage. Finally, it should also be designed to prevent malicious nodes from cooperating to improve each other’s reputation or harm the

reputation of other nodes.

5 Auction Mechanisms

Originally, a solution similar to Peermart [5] was considered due to its algorithmic agnostic properties. A simpler solution was instead followed that follows the English Auction model.

The auction system allows nodes to list items for auction, which all nodes in the network can then subscribe to and bid on. After the item is sold, it should be marked as sold and no longer available for bidding. The node which initiates the auction process by advertising an item to sell, is responsible from determining the who the buyer is – the node with the highest bid – and, upon completion of the auction, request the agreed payment.

6 Conclusion

In this report, we discussed the implementation of a decentralized public ledger that supports both PoW and PoS as the consensus mechanism. Regarding the P2P network, we implemented we integrated Kademlia as the routing protocol for efficient and reliable information exchange among peers, and implemented protections against Sybil and Eclipse attacks. We also implemented auction mechanisms that stores transactions on the blockchain. Finally, we also incorporated a publisher-subscriber model on top of the Kademlia network to support the auction system.

Unfortunately, due to time constraints, we were not able to test our application using Google Cloud’s virtual machines. Nevertheless, we believe that with enough time, these flaws could be addressed without difficulty.

Overall, this practical assignment provided us with valuable insights into the challenges of implementing security in a complex system, as well as an understanding of the intricacies of building a P2P network.

References

- [1] Petar Maymounkov and David Mazieres. Kademlia: A peer-to-peer information system based on the XOR metric. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers*, pages 53–65. Springer, 2002.
- [2] John R Douceur. The sybil attack. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers 1*, pages 251–260. Springer, 2002.
- [3] Riccardo Pecori. S-Kademlia: A trust and reputation method to mitigate a Sybil attack in Kademlia. *Computer Networks*, 94:205–218, 2016.
- [4] Bitcoin Developer. Transactions. <https://developer.bitcoin.org/devguide/transactions.html>.
- [5] David Hausheer and Burkhard Stiller. Peermart: The technology for a distributed auction-based market for peer-to-peer services. In *IEEE International Conference on Communications, 2005. ICC 2005. 2005*, volume 3, pages 1583–1587. IEEE, 2005.